FACT: Fact-Augmented Claim Tester for Detecting Hallucination in LLMs

Min Ko
mk9803@g.rit.edu
Rochester Institute of Technology
Rochester, New York, USA

ACM Reference Format:
Min Ko. . FACT: Fact-Augmented Claim Tester for Detecting Hallucination in LLMs. In . ACM, New York, NY, USA, 7 pages.

1 Introduction

Large Language Models (LLMs) have shown great ability to generate coherent, well-written text. Recently, reasoning models like GPT-5 and Gemini 2.5 Pro have even integrated "thinking" into their generation, performing a long chain of thought before creating a final response. These advancements have improved LLMs' ability to "think" through complex prompts to generate a seemingly correct and well-thought-out answers, encouraging more adoption of LLM use in both industry and daily life [18].

Despite these advancements, hallucinations, or when LLMs generate responses that sound correct but are factually-incorrect, remains a persistent and critical challenge [6]. Research by OpenAI have shown that their reasoning models such as gpt-5-thinking and OpenAI o-mini in fact hallucinate more than the previous non-reasoning models [11]. As a user, it is difficult to trust these responses and requires manual fact checking. Existing solutions such as Retrieval-Augmented Generation (RAG) [8] support LLMs by providing information to base the responses on during generation, but fails to verify the final responses as factually-correct. This lack of after-generation verification justifies the need for a pipeline that combines fact extraction, evidence retrieval and fact verification to detect LLM hallucinations with supporting evidence.

The fact-checking pipeline consists of four stages that take a LLM response and outputs any hallucinated fact with evidence:

1. **Fact Extraction**: Extract atomic facts from a LLM response
2. **Knowledge Base**: Compile trusted sources to create the central knowledge base
3. **Evidence Retrieval**: Use extracted facts as queries to knowledge base and retrieve supporting evidence
4. **Fact Verification**: Classify extracted facts as *supported*, *refuted*, or *unverifiable* using retrieved evidence.

Fact extraction is implemented using Google's Flan-T5 (Fine-tuned LAnguage Net T5) model [3] that is fine-tuned to this specific task using the Fact Extraction and VERification (FEVER) dataset [16]. The FEVER dataset contains individual facts extracted from Wikipedia which are manually annotated as "Supported", "Refuted" or "NotEnoughInfo" with corresponding evidence. The knowledge base is built as a hybrid database with both dense and sparse representations on trusted truth sources such as Wikipedia. As such sources may not have information that is recent or obscure, a web searching API updates the knowledge base with new online information from selected domains if relevant information is not available. Using the extracted facts as queries, supporting evidence is retrieved using two-stage retrieval and a reranker model to find the best matched evidence [13]. Finally, fact verification uses a fine-tuned NLI model that determines whether the evidence supports or refutes the corresponding fact and displays the evidence.

To evaluate FACT, a custom benchmark is built as there are no existing datasets that have all gold, or true, extracted facts with corresponding evidence and labels. Using this benchmark, the pipeline is evaluated to determine the precision, recall and F1 scores for the fact extraction and fact verification stages. We evaluate only the recall score for the evidence retrieval stage as this stage returns a set of documents that may include extra evidence which does not negatively affect the fact verification stage. FACT achieves reasonable F1 scores of 0.745 for fact extraction and 0.774 for fact verification with the weakest stage being evidence retrieval with a recall score of 0.68. Although the results of the stages are not as expected, FACT presents a clear, modular framework to detect hallucinations from any given LLM output where the stages can be individually updated to improve the entire pipeline.

This paper is organized as follows. Section 2 presents the proposed pipeline. Section 3 discusses the experiments performed for each pipeline stage. Section 4 presents the evaluation results. Section 5 discuss key findings and future work. Section 6 concludes the paper.

2 FACT Pipeline

An overview of the proposed fact-checking pipeline is shown in Figure 1.

RIT Computer Science, 20251
.

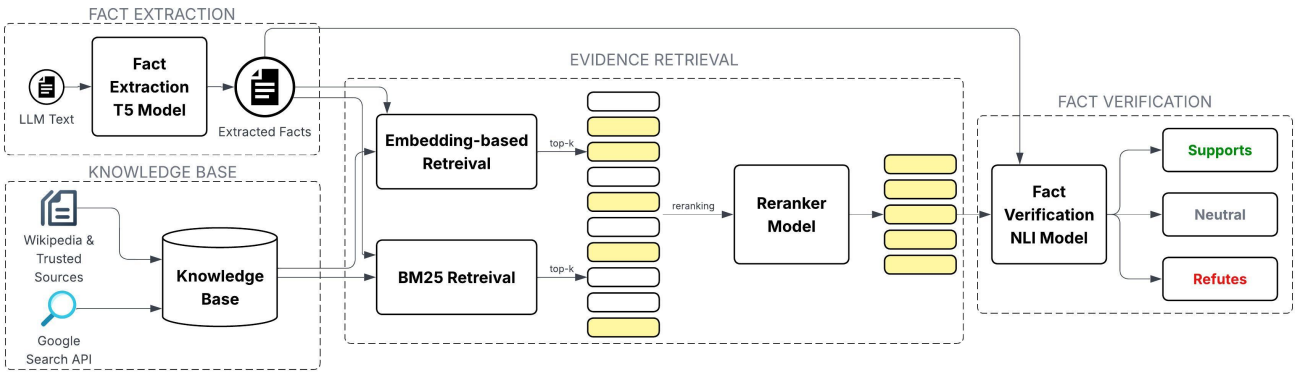

Figure 1. FACT Pipeline

2.1 Fact Extraction

The first stage of the pipeline decomposes LLM outputs into atomic claims, where each claim representing a single, verifiable fact. To achieve this, we use Google's Flan-T5 transformer model [3], fine-tuned specifically for fact extraction.

The T5 model [12] was developed to test the limits of transfer learning in Natural Language Processing (NLP) by introducing a framework that converted all text-related task into a text-to-text format and eliminated the need to train separate, task-specific models. Trained on the Colossal Clean Crawled Corpus (C4) dataset with 745GB of text, the T5 model has a deep and varied understanding of English that is effectively transferred to new, specific tasks through fine-tuning. As a result, it achieved performance comparable to or exceeding state-of-the-art models at the time across multiple NLP benchmarks. Flan-T5 improves upon this framework by using a more advanced instruction-finetuning process, introducing a wider variety of task instructions and achieving up to 25% increase for certain benchmarks [3].

To finetune this model for fact extraction, we use the FEVER dataset [16], which contains facts that are annotated as "Supported", "Refuted" or "NotEnoughInfo" with the corresponding evidence from Wikipedia. This data is further processed by merging facts that corresponds to the same evidence to ensure that the evidence is decomposed completely into facts. This step enables the model to handle a wide variety of input and output text size, avoiding bias towards extracting a certain number of facts.

To supplement FEVER, a supporting dataset of 5,000 LLM-generated text is added to increase the variety of the data. The LLM outputs are split into atomic claims and manually annotated to ensure the quality, helping the model generalize to the style of LLM-generated content.

The Flan-T5 model is then trained on this combined dataset, which balances high-quality human-annotated facts with diverse, realistic LLM outputs, resulting in a fine-tuned model capable of reliably extracting verifiable claims from complex LLM responses.

2.2 Knowledge Base

The second stage of the pipeline builds the initial knowledge base and dynamically adds information that is not present. Text from trusted ground truth sources, such as Wikipedia, are compiled as documents as the initial knowledge base. As recent or niche topics might not be present in such sources, the pipeline uses Google Custom Search JSON API to retrieve reliable information from trusted providers and dynamically updates the knowledge base.

All documents are preprocessed and stored to support hybrid retrieval in stage 3. The documents are split into approximately 500-character chunks without splitting words or sentences in half to maintain the semantic of each chunk. Each chunk is encoded using a sentence transformer [14] model into fixed-length vectors, creating dense representations that capture the semantic, and stored in a vector store, ChromaDB. In parallel, all chunks are also indexed using BM25 [15], a probabilistic keyword-based scoring algorithm that measures the relevance of a chunk to a given query, to create sparse representations that focus on key term frequencies. By storing both dense and sparse representations, the knowledge base supports retrieval using semantic similarity and textual overlap for a more varied list of evidence used later in the pipeline.

2.3 Evidence Retrieval

The third stage of the pipeline retrieves supporting evidence from the knowledge base to verify the extracted facts from stage 1. The retrieval process uses the two-stage retrieval framework that is widely used in modern RAG systems and popularize by Karpukhin et al. [7, 8, 10].

In the first stage, hybrid retrieval is used to get candidate evidence from the knowledge base using the extracted facts

as queries. Dense retrieval encodes the query using a sentence transformer [14] and retrieves its top-k candidates using cosine similarity. To complement this, sparse retriever uses the query to retrieve its top-k candidates with exact keyword matches based on BM25 score.

Dense retrieval provides high recall by identifying contextually relevant evidence even when the query and source text use different wording, while BM25 provides high precision by emphasizing exact lexical overlap. The top candidates from both retrievers are merged to form a combined set for reranking.

In the second stage, the candidate set is passed to a cross-encoder reranker based on a transformer model [10]. This model jointly encodes the query with each candidate evidence, leveraging self-attention to evaluate token-level relationships between the query and evidence. This allows the reranker to distinguish evidence that are truly relevant to the query, providing a better ranking among the candidates to ensure that the most relevant set of evidence is prioritized for the final stage of the pipeline.

2.4 Fact Verification

The fourth stage of the pipeline verifies each extracted fact with the retrieved set of evidence using a natural language inference (NLI) model. The NLI task takes a premise and a hypothesis to classify their relationship as *entailment*, *contradiction* or *neutral*. In the context of fact verification, the extracted fact is the hypothesis and the retrieved evidence is the premise.

We use a transformer-based DeBERTa model fine-tuned for NLI tasks on the large-scale, high-quality datasets such as SNLI [2] and MultiNLI [17] datasets. To adapt the model specifically for factual verification, we further fine-tune it on the previously processed FEVER dataset [16].

The verification stage operates as follows. For each fact, each of the candidate evidence from the third stage is encoded together with the fact and passed through the NLI model. The model outputs a confidence score for each of the three NLI labels, indicating the degree to which the evidence supports, refutes, or is unrelated to the fact. The pipeline then aggregates these confidence scores across all evidence to produce a final verification label and confidence score for the fact.

This stage completes the pipeline, allowing it to take an LLM output, decompose it into individual facts, and produce a final evidence-backed verification of each fact.

Claim: *The Rodney King riots took place in the most populous county in the USA.*

[wiki/Los_Angeles_Riots]
The 1992 Los Angeles riots, also known as the Rodney King riots, were a series of riots, lootings, arsons, and civil disturbances that occurred in Los Angeles County, California in April and May 1992.

[wiki/Los_Angeles_County]
Los Angeles County, officially the County of Los Angeles, is the most populous county in the USA.

Verdict: Supported

Figure 2. Example claim with Wikipedia evidence from original FEVER paper [16].

Source Document
Seventy years ago, Anne Frank died of typhus in a Nazi concentration camp at the age of 15. Just two weeks after her supposed death on March 31, 1945, the Bergen-Belsen concentration camp where she had been imprisoned was liberated…

Right Summary
Anne Frank died earlier than previously believed. Researchers re-examined archives and testimonies of survivors. Anne and older sister Margot Frank are believed to have died in February 1945.

Hallucinated Summary
Anne Frank and her sister Margot were found alive in the Bergen-Belsen concentration camp after new research was released by the Anne Frank House. Witnesses at the camp stated that they saw the sisters alive as late as February 7…

Figure 3. Example hallucinated ChatGPT response from HaluEval [9].

3 Experiments

3.1 Experimental Setup

We conduct a series of experiments for each component of the pipeline to determine the best performing models and approaches to build the FACT pipeline. All experiments are performed on a local laptop with a Intel Core i9-13900HX CPU, NVIDIA RTX 4070 Laptop GPU and 32 GB of RAM.

Source Document
Centre-back Anderson, 27, has cancelled his Barrow contract by mutual consent to join their National League rivals. The ex-Aldershot and Exeter player, who has also played in Italy, could feature against his old club on 18 February. Striker Keating, 21, was recommended to Gulls boss Kevin Nicholson by ex-Port Vale manager Micky Adams…

LLM Summary
Anderson, a 27-year-old center-back, has left Barrow by mutual consent to join their National League rivals. He has previously played for Aldershot, Exeter, and in Italy. He could debut against his former club on February 18. Keating, a 21-year-old striker, is joining Torquay United, subject to international clearance. Gulls boss Kevin Nicholson recommended Keating…

Annotation
Label: Unwanted
Note: Keating was recommended to Kevin by Micky. It's not that Kevin recommended Keating
Summary span: Gulls boss Kevin Nicholson recommended Keating.
Source span: Striker Keating, 21, was recommended to Gulls boss Kevin Nicholson by ex-Port Vale manager Micky Adams.

Figure 4. Example hallucinated LLM response from FaithBench [1].

The FEVER dataset [16] is primarily used for training as it provides a large collection of human-written claims with annotated evidence from Wikipedia sources as shown in Figure 2. Additionally, a custom dataset is built using existing LLM hallucination benchmarks HaluEval [9] and FaithBench [1].

HaluEval includes hallucinated LLM outputs from four different tasks: general, question and answer, summarization and dialogue. We select summarization as it provides longer LLM texts which represent the expected inputs for the FACT pipeline as shown in Figure 3. The hallucinated summary is manually annotated to extract a list of gold claims with gold labels and evidence using the correct summary and source document. FaithBench also provides LLM summaries with the corresponding source documents. Additionally, human annotations of possible hallucinations in the summary are given with notes detailing the reason and supporting source evidence as shown in Figure 4. Each annotation is examined to determine the validity of the hallucination and manually rewritten to create a list of gold claims with gold labels and evidence similarly to HaluEval. It is important to note that these claims are both true and false so the the pipeline is able to extract all claims made by the LLM to then verify with the evidence. 5,000 samples from HaluEval and 459 samples from FaithBench were manually processed and combined to created the final custom dataset, 10% of which is set aside for evaluation. The remaining 90% is combined with FEVER for training the various models.

For evaluation, we chose precision, recall and F1 score for the fact extraction and fact verification stage. These metrics represent different behaviors of machine learning models that are important to understand the strengths and weaknesses. Precision measures how many of the models' predictions are true which is important to prioritize when false positives are undesired. In contrast, recall measures how many of the actual ground truths are predicted correctly and is prioritized when false negatives are more costly. F1 score represents the harmonic mean of precision and recall to provide a general performance measure of the model's performance. For the evidence retrieval stage, we only evaluate the recall score as this stage returns a set of relevant evidence while the evaluation dataset contains only one evidence If the gold evidence is in the set of retrieved evidence, it is considered a success as other valid evidence can exist in the database that are not labeled as the truth. Table 1 shows the metrics of the best performing approach for each stage.

3.2 Fact Extraction

The raw FEVER dataset format is not suitable for this task as each sample is only a single claim. Training Google's Flan-T5 model on this dataset results in the model only able to extract one claim per LLM input regardless of the length. To improve this, FEVER is processed to combine claims that correspond to the same piece of evidence together. Using this approach, one sample evidence text has a list of varying length that contains the corresponding claims, helping the model learn to extract a wide variety of output claims.

Figure 5 shows the performance comparison of models trained on the different training datasets. Using the processed FEVER dataset demonstrates a big improvement over the base version with an increase in F1 score of 0.27. Recall is lower for the base FEVER model as the model extracts only one claim per LLM input regardless of whether multiple claims are expected. With the processed FEVER model, precision is lower as we observed that the model is extracted more claims than we annotated for the benchmark because compound sentences are split into individual claims. Although this is the desired behavior, it is not clearly reflected in the evaluation strategy as we use our fact verification NLI model to determine if the extracted and gold claims support one

Stage	Precision	Recall	F1 Score
Fact Extraction	0.673	0.836	0.745
Retrieval Stage 1 (Hybrid)	–	0.74	–
Retrieval Stage 2 (Reranker)	–	0.68	–
Fact Verification (Overalll)	0.759	0.791	0.774
SUPPORTS	0.792	0.816	0.804
REFUTES	0.721	0.760	0.740

Table 1. FACT evaluation metrics for fact extraction, two-stage retrieval, and fact verification. Retrieval stages do not compute precision/F1.

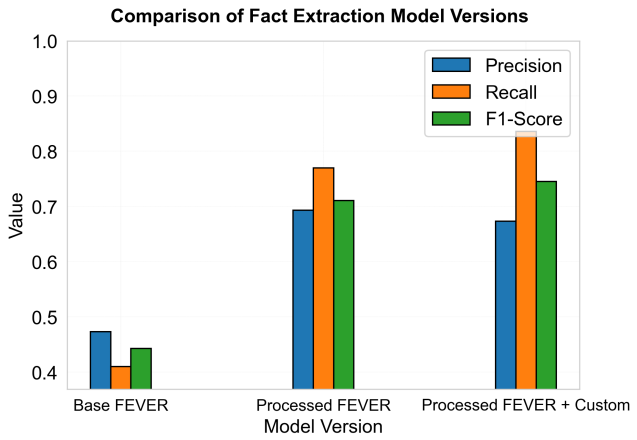

Figure 5. Comparison of fact extraction models trained on different dataset versions.

another. This approach can be flawed if the extract claim lack some information present in the gold claim, leading to a lower score. Regardless, the high recall score shows that the model is able to detect more of the gold claims which is important to detect all possible hallucination in the LLM output. Adding the custom dataset to the model lowers the precision but improves the recall by a higher amount, leading to the better overall F1 score. From the experiments, this configuration emerges as the best performing model for this project.

3.3 Knowledge Base and Evidence Retrieval

The experiments for the knowledge base and evidence retrievals are combined here as they are closely tied together. We use a sentence transformer [14], specifically the *all-mpnet-base-v2* model from Hugging Face [4], to create embeddings for the source documents, which are dense representations that capture the semantic of the document in fixed-length vectors. To store these, we use a vector store called ChromaDB that is designed for machine learning use with built-in functionality for different similarity metrics. Cosine similarity is chosen to be used as the standard metric since it can effectively measure the semantic similarity between two vectors regardless of their length.

The initial version for these stages use this approach to retrieve the evidence but we found that it was not effective in retrieving the correct documents. To improve this, two-stage retrieval is explored to add more coverage in the similarity check by introducing a sparse representation of the documents using the BM25 algorithm [15]. BM25 uses a probabilistic keyword-based matching score to determine the similarity score. This hybrid approach allows the pipeline to retrieve documents based on both the semantic similarity and the textual overlap, returning a more comprehensive list of possible evidence. The second stage of two-stage retrieval introduces a cross-encoder reranker model [10], specifically the DeBERTa-based *nli-deberta-v3-large* model from Hugging Face [5], to rank these retrieved documents to determine the final set of most relevant evidence. This cross-encoder model excels at this task as it jointly encodes the extracted fact and a retrieved document and scores their relevance together instead of encoding each sentence separately. This joint encoding allows the model to have full attention across all tokens to capture subtle relational cues that might be missed by separate encoder models. As a result, the reranker is able to reduce the list of documents to a final set of documents that is most relevant to the given fact.

As the extracted facts are used as queries for the evidence retrieval, there is no guarantee that the extracted facts are valid or hallucinations. Using hallucinations as queries may negatively affect the retrieved documents as the wrong semantic will result in a lower similarity score. To test this, we use a small LLM to rewrite the extracted fact as a neutral question that the fact can answer. By doing this, the semantics of the query are transformed into questions that assumes no truths about the subject. For the experiment, there are four configurations for two-stage retrieval:

1. Hybrid Retrieval (Original Query) + Reranker (Original Query)
2. Hybrid Retrieval (Original Query) + Reranker (Re-written Query)
3. Hybrid Retrieval (Re-written Query) + Reranker (Original Query)

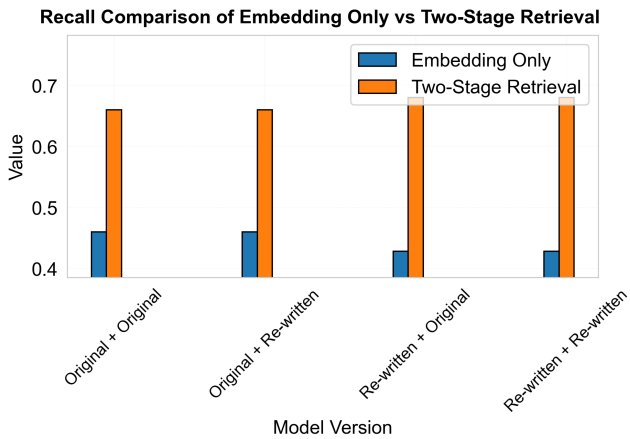

Figure 6. Recall comparison of retrieval strategies with different query re-writing configurations.

4. Hybrid Retrieval (Re-written Query) + Reranker (Re-written Query)

The embedding only approach follows the query plan for the hybrid retrieval.

Figure 6 shows that using two-stage retrieval significantly increases recall compared to using only the embeddings with an average increase of 0.22. For query re-writing, there is no significant difference in performance, suggesting that the retrieval strategies are not significantly affected by hallucinated semantics in the queries. The embedding only approach's performance decreased when the query was re-written, suggesting that the query transformation loses some important semantic. The two-stage approach's best configuration is when the hybrid retrieval uses the re-written query with the reranker showing no clear preference for which query to use, suggesting that the BM25 algorithm benefits from re-writing the query to remove any false assumptions. As such, the pipeline uses the two-stage retrieval method with re-written query for hybrid retrieval and the original query for the reranker.

3.4 Fact Verification

[TO DO]

4 End-to-End Evaluation

[TO DO]

$$\text{E2E Weighted Score} = \frac{\text{\# correctly extracted and verified claims}}{\text{\# total claims}}$$

5 Future Work

Our evaluations show that although each component of FACT provides reasonable results, there are improvements to be made to both the evaluation strategy and the components. The correctness for fact extraction was determined using the NLI model to check whether the predicted fact and the gold fact supported one another. This approach solely relies on the NLI model used as it directly affects the accuracy of the evaluation itself. As an improvement, a more dependable evaluation method is needed to determine whether an extracted fact matches the gold fact.

Class	E2E Weighted Score
SUPPORTS	0.613
REFUTES	0.694
Overall	0.643

Table 2. End-to-end evaluation metrics for the FACT pipeline.

As for the pipeline, each stage could be improved by introducing recent advancements in their respective fields and leveraging LLMs carefully . For fact extraction, structured LLM prompting and language modeling can be used to extract more accurate claims from complex text that contains multiple entities, nested relations, or implicit references. An additional step could introduce multi-granular extraction to get claims from the sentence, paragraph, and document level for more coverage and less redundancy.

For evidence retrieval, more extensive strategies such as multi-hop and graph-based retrieval could be explored to connect supporting documents that exists in different sources. Multi-hop retrieval allows the system to iteratively reason through the extracted fact to retrieve evidence on connected documents that can provide a more coherent context for verification. Graph-based retrieval can enhance this by creating structured entity–relation graphs to locate evidence that is connected through complex relational chains.

For fact verification, recent advances in LLM-based fact checking can be explored to determine their ability to make trustworthy classification.

6 Conclusion

In this paper, we have presented FACT, an end-to-end fact-checking pipeline designed to check LLM outputs for hallucinations that are grounded with clear evidence. There are four distinct stages: fact extraction, knowledge base, evidence retrieval and fact verification. The fact extraction stage takes LLM outputs to break down into individual facts needed to be verified. The knowledge base stage compiles trusted sources, such as Wikipedia, into a truth database with the ability to dynamically update itself using web search API on more recent topics. The evidence retrieval stage uses the set of extract facts from the first stage as queries for two-stage retrieval into the knowledge base to get the most relevant evidence for each fact. Finally, the fact verification stage determines whether the evidence supports or refutes the fact

that was made by the LLM. As a result, FACT presents a complete pipeline that takes full LLM outputs and classify hallucinations made with the corresponding evidence. Our evaluations show that FACT achieves an end-to-end correctly extracted and verified claim accuracy of 0.64 with the fact extraction stage evaluation being flawed. Future work can improve the evaluation strategy and integrate modern advancements to the components to enhance their individual performance.

References

[1] Forrest Bao, Miaoran Li, Renyi Qu, Ge Luo, Erana Wan, Yujia Tang, Weisi Fan, Manveer Singh Tamber, Suleman Kazi, Vivek Sourabh, et al. 2025. Faithbench: A diverse hallucination benchmark for summarization by modern llms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*. 448–461.

[2] Samuel R Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. 2015. A large annotated corpus for learning natural language inference. *arXiv preprint arXiv:1508.05326* (2015).

[3] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[4] Hugging Face. 2021. sentence-transformers/all-mpnet-base-v2. https://huggingface.co/sentence-transformers/all-mpnet-base-v2. Hugging Face model repository, accessed 2025-12-08.

[5] Hugging Face. 2024. cross-encoder/nli-deberta-v3-large: A Natural Language Inference model. https://huggingface.co/cross-encoder/nli-deberta-v3-large. Hugging Face model repository, accessed 2025-12-08.

[6] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.

[7] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering.. In *EMNLP (1)*. 6769–6781.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[9] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. Halueval: A large-scale hallucination evaluation benchmark for large language models. *arXiv preprint arXiv:2305.11747* (2023).

[10] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085* (2019).

[11] OpenAI. 2025. *GPT-5 System Card*. https://openai.com/index/gpt-5-system-card/ Accessed: 2025-10-26.

[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[13] Jhon Rayo, Raúl de La Rosa, and Mario Garrido. 2025. A Hybrid Approach to Information Retrieval and Answer Generation for Regulatory Texts. *arXiv preprint arXiv:2502.16767* (2025).

[14] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[15] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[16] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and VERification. *arXiv preprint arXiv:1803.05355* (2018).

[17] Adina Williams, Nikita Nangia, and Samuel R Bowman. 2017. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426* (2017).

[18] Fengli Xu, Qianyue Hao, Chenyang Shao, Zefang Zong, Yu Li, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, et al. 2025. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns* 6, 10 (2025).